

Chương 5.

Xử lý request và response

- 5.1. Kiểm tra và validate dữ liệu đầu vào
- 5.2. Xử lý lỗi và exception
- 5.3. Thiết lập quy tắc response
- 5.4. Thực hành

- Phương pháp gửi dữ liệu từ client về server
- Cách thức nhận dữ liệu trên máy chủ web
- Sử dụng các attribute để binding dữ liệu từ request
- Validate dữ liệu đầu vào bằng Data Annotations

Các phương pháp thường dùng:

- **GET:** Gửi dữ liệu qua query string (dữ liệu hiển thị trên URL).
- **POST:** Gửi dữ liệu trong body của request (phổ biến cho form hoặc API).
- **PUT/PATCH:** Sử dụng để cập nhật dữ liệu.
- **DELETE:** Dùng để xóa dữ liệu.

- Dữ liệu thường được gửi dưới dạng:
 - ✓ Query Parameters (GET): <https://api.example.com/users?id=1>
 - ✓ JSON Body (POST/PUT): { "username": "user1", "email": "user1@example.com" }

- Spring Boot hỗ trợ binding dữ liệu từ request vào các đối tượng Java.
- ✓ Sử dụng annotation `@RequestBody` để binding JSON vào model.
- ✓ `@RequestParam` hoặc `@PathVariable` cho query hoặc path parameter.

Sử dụng các attribute để binding dữ liệu từ request

- Spring Boot hỗ trợ nhiều annotation để binding dữ liệu:
 - ✓ `@RequestBody`: Binding JSON từ body của request vào object.
 - ✓ `@RequestParam`: Binding dữ liệu từ query parameter.
 - ✓ `@PathVariable`: Binding dữ liệu từ URL path.

- **Spring Boot sử dụng thư viện Hibernate Validator để cung cấp các annotation cho validate.**
- Ví dụ validate model:

- Kiểm tra lỗi validate với BindingResult:
 - Nếu có lỗi, các thông tin lỗi sẽ được lưu trong đối tượng BindingResult.
- Ví dụ trả lỗi validate:

Ví dụ các tình huống validate

- Username trống: "Username is required"
- Email sai định dạng: "Email should be valid"
- Username dưới 3 ký tự: "Username must be between 3 and 50 characters"

1. Tạo API quản lý sản phẩm:

- Tạo API POST /api/products để thêm mới sản phẩm.
- Model sản phẩm có các trường:
 - ✓ name: Bắt buộc, tối thiểu 3 ký tự.
 - ✓ price: Bắt buộc, giá trị phải lớn hơn 0.

2. Xử lý lỗi validate:

Trả về danh sách lỗi nếu dữ liệu không hợp lệ.

3. Mở rộng:

- Thêm API GET /api/products/{id} để lấy thông tin sản phẩm theo ID.
- Thêm validate cho id (phải là số nguyên dương).

1. Sử dụng try-catch để xử lý exception
2. Trả về lỗi HTTP: BadRequest, NotFound, Unauthorized
3. Ghi log lỗi bằng SLF4J và Logback
4. Cấu hình trang lỗi chung với @ControllerAdvice
5. Cấu hình middleware xử lý lỗi

1. Sử dụng try-catch để xử lý exception

- **Cách xử lý lỗi với try-catch:**

- ✓ try-catch dùng để bắt các lỗi runtime trong ứng dụng.

- ✓ Ví dụ cơ bản:

```
public int divide(int a, int b) {  
    try {  
        return a / b;  
    } catch (ArithmeticException e) {  
        System.out.println("Error: Division by zero");  
        return 0; // Trả về giá trị mặc định  
    }  
}
```

2. Trả về lỗi HTTP: BadRequest, NotFound, Unauthorized

- Trả về mã lỗi HTTP: Sử dụng các phương thức của ResponseEntity như:
 - ✓ `ResponseEntity.badRequest()`
 - ✓ `ResponseEntity.status(HttpStatus.NOT_FOUND)`
 - ✓ `ResponseEntity.status(HttpStatus.UNAUTHORIZED)`
- Ví dụ:

3. Ghi log lỗi bằng SLF4J và Logback

- Cài đặt logging trong Spring Boot:
 - ✓ Spring Boot tích hợp sẵn SLF4J và Logback.
 - ✓ Cấu hình trong file application.properties:
logging.level.root=INFO
logging.level.com.example=DEBUG
logging.file.name=logs/application.log

Cấu hình trang lỗi chung với @ControllerAdvice

- Tạo lớp xử lý lỗi toàn cục:

```
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ControllerAdvice;
import org.springframework.web.bind.annotation.ExceptionHandler;

@ControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(IllegalArgumentException.class)
    public ResponseEntity<?> handleIllegalArgumentException(IllegalArgumentException ex)
    {
        return ResponseEntity.badRequest().body("Invalid input: " + ex.getMessage());
    }
    @ExceptionHandler(Exception.class)
    public ResponseEntity<?> handleGenericException(Exception ex) {
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body("Server
error: " + ex.getMessage());
    }
}
```

Cấu hình trang lỗi chung với @ControllerAdvice

- Khi xảy ra lỗi, Controller không cần try-catch:

```
@GetMapping("/{id}")  
public ResponseEntity<?> getProduct(@PathVariable Long id) {  
    if (id == 0) {  
        throw new IllegalArgumentException("Invalid product ID");  
    }  
    return ResponseEntity.ok("Product details for ID: " + id);  
}
```

5. Cấu hình middleware xử lý lỗi

- Sử dụng `ExceptionHandler` để xử lý lỗi chung:

```
import org.springframework.boot.web.servlet.error.ExceptionHandler;  
import org.springframework.web.bind.annotation.RequestMapping;  
import org.springframework.web.bind.annotation.RestController;  
@ExceptionHandler  
public class CustomExceptionHandler implements ExceptionHandler {  
    @RequestMapping("/error")  
    public ResponseEntity<?> handleError() {  
        return  
        ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body("An  
unexpected error occurred");  
    }  
}
```

- Endpoint: /api/math/divide
- Request: /api/math/divide?a=10&b=0
- Response:

```
{  
  "error": "Error: Division by zero"  
}
```

- Tạo API sản phẩm:
 - ✓ Endpoint POST /api/products: Thêm sản phẩm mới.
 - ✓ Validate dữ liệu đầu vào và trả lỗi với mã HTTP tương ứng.
- Xử lý lỗi toàn cục: Xử lý lỗi NullPointerException và trả mã lỗi HTTP 500.
- Ghi log: Ghi log tất cả request và exception vào file log.

1. Phương pháp trả dữ liệu về client

- Trả dữ liệu đơn giản: JSON hoặc XML:

```
return Ok(new { success = true, message = "Data  
processed successfully" });
```

- **Sử dụng DTO (Data Transfer Object):** Đảm bảo dữ liệu trả về nhất quán, tránh lộ thông tin không cần thiết.

```
public class UserDto  
{  
    public string Username { get; set; }  
    public string Email { get; set; }  
}
```

1. Phương pháp trả dữ liệu về client

- **Sử dụng DTO (Data Transfer Object):** Đảm bảo dữ liệu trả về nhất quán, tránh lộ thông tin không cần thiết.

```
public class UserDto
{
    public string Username { get; set; }
    public string Email { get; set; }
}
```

2. Sử dụng [ProducesResponseType]

Cách mô tả các kiểu response

- [**ProducesResponseType**(StatusCodes.Status200K)]
- [**ProducesResponseType**(StatusCodes.Status400BadRequest)]
- [**ProducesResponseType**(StatusCodes.Status404NotFound)]
- `public IActionResult GetUser(int id)`
- `{`
- `// Logic xử lý`
- `}`

3. Trả về dữ liệu bảo mật, tối ưu và cache

- Bảo mật:
 - ✓ Ẩn thông tin nhạy cảm (mã hóa hoặc loại bỏ khỏi response).
 - ✓ Thiết lập CORS policy để kiểm soát quyền truy cập.
- Tối ưu:
 - ✓ Trả về dữ liệu tối thiểu cần thiết.
 - ✓ Sử dụng phân trang hoặc lazy loading khi trả dữ liệu lớn.
- Cache: Sử dụng response caching để cải thiện hiệu suất

3. Trả về dữ liệu bảo mật, tối ưu và cache

- Cache: Sử dụng response caching để cải thiện hiệu suất

```
[ResponseCache(Duration = 60)]
```

```
public IActionResult GetCachedData()  
{  
    return Ok(data);  
}
```

Phần 1: Validate dữ liệu

Viết API nhận thông tin người dùng, validate các trường:

- Username: bắt buộc, 3-50 ký tự.
- Email: định dạng email hợp lệ.
- Password: 8-20 ký tự, chứa ít nhất 1 ký tự đặc biệt.

Phần 2: Xử lý lỗi

Thêm ghi log và trả về lỗi chuẩn trong các API sau:

- API tìm kiếm người dùng (NotFound).
- API cập nhật dữ liệu khi có lỗi server (500).

Phần 3: Thiết lập response

Trả về response dạng JSON cho API quản lý sản phẩm:

- GET /products: danh sách sản phẩm (có phân trang).
- POST /products: thêm mới sản phẩm và trả về ID sản phẩm vừa thêm.



CMC UNIVERSITY

Aspire to Inspire the Digital World

THANK YOU